Neural PDE Solvers Benchmark

A Comprehensive Empirical and Theoretical Evaluation of Physics-Informed Neural Networks, Fourier Neural Operators, and Neural ODEs

Final Cumulative Report

Team Proboticists
Aditya Tyagi & Pranjal Kumar Singh
Mentor: Rakshit Mundra
Machine Learning Research Division

October 6, 2026

Abstract

This document presents the final cumulative findings of an exhaustive empirical and theoretical benchmark comparing three distinct deep learning paradigms for solving Partial Differential Equations (PDEs) and Ordinary Differential Equations (ODEs): Physics-Informed Neural Networks (PINNs), Fourier Neural Operators (FNOs), and Neural Ordinary Differential Equations (Neural ODEs). Using a standardized benchmarking harness (`PDEMeasurementHarness` / `ReconciledMeasurementHarness`), we systematically evaluate each paradigm across three foundational axes: Accuracy (Relative L_2 Error), Computational Cost (Wall-Clock Time), and Amortization Scale. Our findings reveal critical operational boundaries: (1) PINNs suffer from a severe *Advection Cliff* at high wave speeds ($c \geq 20$, Relative L_2 error > 0.92), which supervised regression tests prove to be an optimization bottleneck rather than a representation capacity failure; (2) FNOs demonstrate true resolution invariance with a 65.5% compute savings when upsampling zero-shot from coarse ($n_x = 100$) to fine ($n_x = 400$) grids, but exhibit optimization instability and spectral truncation limits at high frequencies ($k \geq 25$); (3) Neural ODEs achieve high trajectory precision over short horizons ($t \in [0, 5\text{s}]$), but experience exponential error accumulation over long rollout horizons ($t \in [10, 20\text{s}]$) due to the absence of spatial boundary feedback. Finally, we contextualize our results through a rigorous literature audit against Krishnapriyan et al. (2021) and McGreivy & Hakim (2024), providing a definitive decision framework for selecting neural PDE architectures under practical physical and computational constraints.

Contents

1 Executive Summary & Methodological Scope **3**

2 Unified Benchmarking Framework & Amortization Theory **3**
2.1 The Uniform Measurement Harness . . . 3
2.1.1 Hardware Synchronization & Timing Protocol . . . 4
2.1.2 Precision Metric: Epsilon-Safe Relative L_2 Error . . . 4
2.2 Amortization Scaling Mathematics . . . 4
2.2.1 Cost Formulation . . . 4
2.2.2 Amortization Crossover Point . . . 4
2.3 PINN Loss Breakdown & Boundary Integrity . . . 4

3 FNO Continuous Operator Learning & Validation Analysis **5**
3.1 Fourier Neural Operator Architecture . . . 5
3.2 The Continuous Interpolation Validation Experiment . . . 5

3.2.1 Experimental Protocol . . . 5
3.2.2 Architectural Post-Mortem: Over-Capacity and Data Starvation . . . 5

4 Investigation 1: The Advection Cliff and Optimization Bottleneck 6
4.1 Problem Formulation . . . 6
4.2 The Advection Cliff Collapse . . . 6
4.3 Hypothesis Testing: Capacity vs. Optimization Failure . . . 6
4.4 The Supervised "Cheat" Test Proof . . . 6

5 Investigation 2: Water Wave Dynamics Across Depth Regimes 7
5.1 Problem Formulation . . . 7
5.2 Regime Breakdown & PINN Failure Modes . . . 7

6 Investigation 3: FNO Resolution Invariance & Optimization Sensitivity 8
6.1 Zero-Shot Upsampling Evaluation . . . 8
6.2 Computational Advantage Analysis . . . 8
6.3 Secondary Vulnerability: Optimization Initializer Sensitivity . . . 8

7 Investigation 4: Neural ODE Rollout Stability 8
7.1 Experimental Configuration . . . 8
7.2 Integrator & Step Size Sensitivity Analysis . . . 8
7.3 Rollout Failure Modes & Error Compounding . . . 9

8 Stage 3 Literature Audit & Convergence Report 9
8.1 Part 1: Validating PINN Failure Modes (Krishnapriyan et al., 2021) . . . 9
8.2 Part 2: Project Audit & Reporting Biases (McGreivy & Hakim, 2024) . . . 9

9 Architectural Convergence & Decision Framework 10

A Python Reference Code: Fourier Neural Operator (1D) 11

B Python Reference Code: Reconciled Measurement Harness 11

1 Executive Summary & Methodological Scope

Differential equations form the mathematical foundation of physical modeling across fluid mechanics, thermodynamics, wave propagation, and dynamical systems. Traditional numerical solvers—such as Finite Difference, Finite Element, and Pseudo-Spectral methods—offer provable accuracy and stability guarantees but scale exponentially in computational complexity for high-dimensional or parametric problems.

In recent years, deep learning approaches have emerged as compelling alternatives. However, these paradigms embody fundamentally different mathematical formulations, operational assumptions, and computational trade-offs:

- **Physics-Informed Neural Networks (PINNs):** Function approximators that integrate physical conservation laws directly into the loss function via Automatic Differentiation (AD). PINNs operate as *instance-based solvers*, requiring full re-optimization for every new boundary condition, initial state, or parameter variation.
- **Fourier Neural Operators (FNOs):** Data-driven *operator-learning models* that parametrize integral kernels in Fourier space. FNOs map infinite-dimensional function spaces to function spaces, learning the underlying PDE operator over an entire ensemble of initial conditions.
- **Neural Ordinary Differential Equations (Neural ODEs):** Continuous-time dynamical systems that replace discrete hidden layer transitions with continuous vector fields integrated via numerical ODE solvers (e.g., Runge-Kutta 4 or adaptive Dopri5).

This investigation presents an end-to-end empirical comparison of these three frameworks, establishing explicit performance envelopes, failure modes, and amortization crossover thresholds.

Table 1: Methodological Overview of Evaluated Neural Differential Equation Solvers

Feature / Property	**PINN (Instance-Based)**	**FNO (Amortized Operator)**	**Neural ODE (Continuous-Time)**
Primary Input	Spatial-temporal coords (x, t)	Initial state $u_0(x)$ on grid	State vector $z(t)$ at time t
Supervision Type	Unsupervised (PDE Residual)	Supervised (Paired trajectories)	Supervised / Trajectory matching
Amortization	None ($C_{\text{total}} = N \cdot T_{\text{train}}$)	High ($C_{\text{total}} = T_{\text{train}} + N \cdot T_{\text{infer}}$)	Moderate / Parameter-dependent
Grid Reliance	Mesh-free / Continuous	Resolution-invariant grid	Time-continuous / Discrete space
Core Mechanism	AutoDiff + Physics Loss	Fast Fourier Transform (FFT)	Adaptive/Fixed ODE Solvers

2 Unified Benchmarking Framework & Amortization Theory

2.1 The Uniform Measurement Harness

To eliminate profiling disparities across heterogeneous PyTorch architectures, all experiments were benchmarked using a single, unified evaluation architecture: the `ReconciledMeasurementHarness`. The harness isolates hardware-level timing artifacts and establishes standardized scoring protocols.

2.1.1 Hardware Synchronization & Timing Protocol

To accurately measure wall-clock execution time across asynchronous GPU execution pipelines, all time measurements enforce explicit CUDA stream synchronization:

$$T_{\text{sync}} = \texttt{torch.cuda.synchronize()} \longrightarrow t_{\text{perf_counter}} \tag{1}$$

2.1.2 Precision Metric: Epsilon-Safe Relative L_2 Error

Accuracy across all benchmarked solutions is evaluated against exact closed-form analytical solutions u_{exact} using an epsilon-safe Relative L_2 error metric:

$$\text{Rel } L_2(u_{\text{pred}}, u_{\text{exact}}) = \frac{\|u_{\text{pred}} - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2 + 10^{-8}} = \frac{\sqrt{\sum_{i=1}^{M}(u_{\text{pred},i} - u_{\text{exact},i})^2}}{\sqrt{\sum_{i=1}^{M} u_{\text{exact},i}^2} + 10^{-8}} \tag{2}$$

2.2 Amortization Scaling Mathematics

A central contribution of our framework is the mathematical distinction between *instance-based optimization* and *amortized operator inference*.

2.2.1 Cost Formulation

For a target problem requiring N distinct physical instances (e.g., N different initial conditions or parameter variations):

$$\text{Total Cost}_{\text{PINN}}(N) = N \cdot T_{\text{train_pinn}} \tag{3}$$

$$\text{Total Cost}_{\text{FNO}}(N) = T_{\text{train_fno}} + N \cdot T_{\text{infer_fno}} \tag{4}$$

where $T_{\text{train_pinn}} \approx 80.0$ seconds per instance, $T_{\text{train_fno}} \approx 230.0$ seconds for the initial dataset generation and training, and $T_{\text{infer_fno}} \approx 0.005$ seconds per instance.

2.2.2 Amortization Crossover Point

The crossover instance count N^* where FNO becomes computationally superior to PINN is derived by equating Equations (3) and (4):

$$N^* = \left\lceil \frac{T_{\text{train_fno}}}{T_{\text{train_pinn}} - T_{\text{infer_fno}}} \right\rceil = \left\lceil \frac{230.0}{80.0 - 0.005} \right\rceil = 3 \text{ instances} \tag{5}$$

Key Amortization Finding

If a practitioner requires solving $N \leq 2$ instances, PINNs are computationally cheaper because they bypass the heavy offline dataset generation phase. However, for $N \geq 3$, FNO amortizes its training cost, delivering up to $10,000\times$ faster per-instance solutions.

2.3 PINN Loss Breakdown & Boundary Integrity

PINNs often suffer from "lazy optimization," where the network minimizes the global PDE residual L_{res} by ignoring localized boundary conditions L_{bc} or initial conditions L_{ic}. Our harness enforces strict loss decomposition profiling:

$$L_{\text{total}} = w_{\text{res}} L_{\text{res}} + w_{\text{bc}} L_{\text{bc}} + w_{\text{ic}} L_{\text{ic}} \tag{6}$$

where $L_{\text{res}} = \frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{D}[u](x_i, t_i) - f(x_i, t_i)|^2$.

3 FNO Continuous Operator Learning & Validation Analysis

3.1 Fourier Neural Operator Architecture

The Fourier Neural Operator models continuous differential operators by lifting input fields into higher-dimensional channel representations, applying iterative Fourier layer updates, and projecting back to physical space:

$$v_{t+1}(x) = \sigma\left(W v_t(x) + \mathcal{F}^{-1}\left(R \cdot \mathcal{F}(v_t)\right)(x)\right) \tag{7}$$

where $\mathcal{F}$ and $\mathcal{F}^{-1}$ denote the Fast Fourier Transform and its inverse, R represents learned complex-valued tensor weights filtering low-frequency Fourier modes up to cut-off $k_{\max}$, W is a local linear transformation, and σ is a non-linear activation function (GELU).

3.2 The Continuous Interpolation Validation Experiment

To prove that the FNO learns a continuous physical operator rather than functioning as a non-linear lookup table that memorizes training samples, a rigorous validation experiment was conducted.

3.2.1 Experimental Protocol

1. **Dataset Construction:** A synthetic training set of 2,000 1D Heat Equation simulations was constructed using *integer wave frequencies* $k \in \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
2. **Out-of-Distribution Test State:** An unseen test state was generated using a fractional frequency $k = 4.5$, representing a physics state strictly outside the training manifold.
3. **Nearest-Neighbour Baseline:** A standard L_2 Nearest-Neighbour model searched the 2,000-sample training memory bank to find the closest match ($k = 4$ or $k = 5$) and returned its output as a blind prediction.
4. **Validation Criterion:** The FNO passes if and only if Rel L_2(FNO) $<$ Rel L_2(Nearest-Neighbour).

Table 2: Validation Benchmark Results on Out-of-Distribution State ($k = 4.5$)

Method	**Rel L_2 Error**	**Interpolation Status**	**Verdict**
Nearest-Neighbour Baseline ($k = 4$ or 5)	0.4125	Failed (Pure Lookup)	Baseline
Initial High-Capacity FNO ($k_{\max} = 32$)	0.4510	Overfitted Memory Bank	**FAILED**
Corrected Regularized FNO ($k_{\max} = 16$)	**0.0382**	**Continuous Operator Learning**	**PASSED**

3.2.2 Architectural Post-Mortem: Over-Capacity and Data Starvation

Our initial FNO implementation failed the validation check, producing an error (0.4510) worse than the naive baseline. Diagnostic analysis revealed two root causes:

- **Excess Fourier Modes ($k_{\max} = 32$):** Smooth physical diffusion in the heat equation is governed by low-frequency modes. Excess high-frequency parameters allowed the network to memorize high-frequency training noise rather than learning the continuous kernel.
- **Data Starvation:** Training high-capacity modes on a restricted frequency grid resulted in severe overfitting. Truncating modes to $k_{\max} = 16$ forced the network to interpolate smoothly across fractional frequencies.

4 Investigation 1: The Advection Cliff and Optimization Bottleneck

4.1 Problem Formulation

We investigate the operational limits of a standard PINN applied to the 1D Advection Equation:

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0, \quad x \in [0,1], \quad t \in [0,1] \tag{8}$$

with periodic boundary conditions and smooth initial condition $u(x,0) = \sin(2\pi x)$. As the wave speed c increases, the temporal gradient becomes sharp, introducing high-frequency spatial-temporal oscillations.

4.2 The Advection Cliff Collapse

We evaluated a 4-layer Multi-Layer Perceptron (MLP) with 64 neurons per layer and Tanh activations across wave speeds $c \in \{1, 5, 20, 50\}$.

Table 3: PINN Performance Spectrum Across Advection Speeds (c)

Wave Speed (c)	**Final Loss (L_{total})**	**Rel L_2 Error**	**Convergence State**
$c = 1$	1.2×10^{-5}	**0.0008**	Perfect Convergence
$c = 5$	4.5×10^{-3}	**0.0209**	Accurate Solution
$c = 20$	0.412	**0.9280**	Systemic Collapse
$c = 50$	0.985	**1.0015**	Trivial Zero Solution (100% Error)

As demonstrated in Table 3, the PINN experiences a catastrophic performance collapse beyond $c = 5$, reaching complete failure at $c = 50$ where it outputs a flat zero solution.

4.3 Hypothesis Testing: Capacity vs. Optimization Failure

To isolate the root cause of the collapse at $c = 50$, two competing hypotheses were evaluated:

1. **Capacity Failure Hypothesis:** The MLP architecture lacks the expressive capacity to represent high-frequency waves moving at $c = 50$.
2. **Optimization Failure Hypothesis:** The neural network possesses sufficient capacity (Universal Approximation Theorem), but the non-convex physics-informed residual loss landscape prevents Adam/L-BFGS optimizers from finding the global minimum.

4.4 The Supervised "Cheat" Test Proof

To test the Expressive Capacity Hypothesis, we conducted a purely supervised regression test. The exact same 4-layer Tanh MLP was trained on dense ground-truth coordinate-solution pairs $(x_i, t_i, u_{\text{exact}}(x_i, t_i))$ for $c = 50$, bypassing the physics residual loss entirely.

Supervised Regression Test Results ($c = 50$)

- **Supervised Final Relative L_2 Error: 0.0208** (2.08% error).
- **Physics-Informed Final Relative L_2 Error: 1.0015** (100.15% error).
- **Definitive Verdict:** The network possesses full mathematical capacity to represent the wave. The collapse of the PINN at $c \geq 20$ is strictly an **Optimization Bottleneck** caused by non-convex physics loss landscapes and gradient pathology.

Furthermore, attempting to resolve this bottleneck by heavily increasing boundary weights ($w_{\text{bc}} = 100, w_{\text{ic}} = 100$) failed to induce convergence, confirming that simple hyperparameter tuning cannot overcome fundamental physics gradient imbalances.

5 Investigation 2: Water Wave Dynamics Across Depth Regimes

5.1 Problem Formulation

Linear water wave dynamics are governed by the velocity potential $\phi(x, z, t)$ satisfying Laplace's equation $\nabla^2\phi = 0$ with linearized free-surface boundary conditions:

$$\phi_0(x, z) = \frac{ag}{\omega} \frac{\cosh(k(z + h))}{\cosh(kh)} \sin(kx) \tag{9}$$

where h is fluid depth, a is wave amplitude, $k = 2\pi/L$ is wave number, and dispersion relation $\omega^2 = gk \tanh(kh)$.

5.2 Regime Breakdown & PINN Failure Modes

We evaluated the PINN across three classic hydrodynamic regimes: Shallow Water ($kh \ll 1$), Intermediate Water ($kh \approx 1$), and Deep Water ($kh \gg 1$).

Table 4: PINN Hydrodynamic Regime Performance Comparison

Regime	Depth Ratio (kh)	Rel L_2 Error	PINN Stability & Wave Extraction
Shallow Water	$kh = 0.10$	**0.0042**	Robust convergence; exact wave speed $c = \sqrt{gh}$
Intermediate Water	$kh = 1.00$	**0.0185**	Accurate potential field tracking
Deep Water	$kh = 10.0$	**0.8740**	**Failure: Hyperbolic cosine explosion/vanish**

Hydrodynamic Failure Analysis

In deep water ($kh \gg 1$), the terms $\cosh(k(z + h))$ and $\cosh(kh)$ grow exponentially ($\approx e^{kh}/2$). Automatic differentiation of these extreme scales produces exploding gradients near the free surface ($z = 0$) and vanishing gradients near the seabed ($z = -h$). The PINN fails to resolve the exponential vertical decay profile.

6 Investigation 3: FNO Resolution Invariance & Optimization Sensitivity

6.1 Zero-Shot Upsampling Evaluation

A primary theoretical claim of Fourier Neural Operators is grid independence. To test this, an FNO was trained on a coarse spatial grid ($n_x = 100$) and evaluated zero-shot on a fine spatial grid ($n_x = 400$) without structural modifications or retraining.

Table 5: FNO Resolution Invariance and Computational Savings

Training Grid → Evaluation Grid	**Mean Rel L_2 Error**	**Training Time (s)**	**Compute Savings**
Native Coarse (100 → 100)	0.2350	37.57 s	Baseline
Zero-Shot Upsampled (100 → 400)	0.3158	37.57 s	**65.5% Savings**
Native Fine (400 → 400)	0.2110	109.01 s	0% (Full Cost)

6.2 Computational Advantage Analysis

As shown in Table 5, training natively on a fine grid ($n_x = 400$) requires 109.01s. By leveraging resolution invariance, a model trained on a cheap coarse grid ($n_x = 100$) in 37.57s can be evaluated on the fine grid with only a modest accuracy penalty (0.2350 → 0.3158). This yields a 65.5% **reduction in computational overhead**.

6.3 Secondary Vulnerability: Optimization Initializer Sensitivity

Across repeated independent training runs under identical hyperparameter settings, FNO performance exhibited severe variance:

$$\text{Rel } L_2 \text{ Error Range} \in [0.0270, 0.5060] \tag{10}$$

This indicates that while FNOs are structurally resolution-invariant, their complex-valued Fourier weight tensors R are highly sensitive to initial random seed assignments, resulting in non-deterministic convergence under standard Adam optimization.

7 Investigation 4: Neural ODE Rollout Stability

7.1 Experimental Configuration

We benchmark continuous-time Neural ODEs driven by a 2-layer MLP (64 neurons) mapping state vector dynamics $\frac{dz}{dt} = f_\theta(z, t)$. The network was trained over a short temporal window $t \in [0, 5\text{s}]$ using RK4 ($dt = 0.05$) and Adam ($lr = 1 \times 10^{-3}$). We analyze error propagation during trajectory rollout extension up to $t = 20$s.

7.2 Integrator & Step Size Sensitivity Analysis

Table 6: Neural ODE Rollout Error Across Temporal Horizons

Integrator	**Step Size (dt)**	**Error [0–5s]**	**Error [5–10s]**	**Error [10–20s]**
Explicit Euler	$dt = 0.05$	0.0120	0.2450	**14.8200 (Exploded)**
Explicit Euler	$dt = 0.01$	0.0035	0.0480	0.8920
Runge-Kutta 4 (RK4)	$dt = 0.05$	**0.0008**	**0.0045**	**0.0210 (Stable)**
Adaptive (Dopri5)	Tolerances 10^{-5}	0.0005	0.0021	0.0095

7.3 Rollout Failure Modes & Error Compounding

Neural ODE rollouts suffer from fundamental failure modes during horizon extension:

- **Autoregressive Error Accumulation:** Because continuous rollouts use previous state outputs $z(t)$ as inputs for subsequent evaluations $z(t + \Delta t)$, minor vector field inaccuracies in f_θ compound exponentially over time:

$$\epsilon(t) \sim \mathcal{O}\left(e^{Lt}\epsilon_0\right) \tag{11}$$

  where L is the Lipschitz constant of f_θ.

- **Lack of Global Spatial Corrections:** Unlike PINNs, which enforce global spatial boundary constraints across the entire domain simultaneously, Neural ODEs lack boundary feedback mechanisms to correct trajectory drift.

- **Early Convergence Sensitivity:** If initial training over $t \in [0, 5\text{s}]$ fails to achieve near-zero loss, trajectory rollouts beyond $t > 10\text{s}$ rapidly diverge to infinity.

8 Stage 3 Literature Audit & Convergence Report

Literature Audit Framework

To maintain scientific rigor, our empirical findings were audited against two seminal literature reviews: Krishnapriyan et al. (2021) *"Characterizing possible failure modes in physics-informed neural networks"* and McGreivy & Hakim (2024) *"Of course neural networks can fit arbitrary curves, but can they solve PDEs?"*.

8.1 Part 1: Validating PINN Failure Modes (Krishnapriyan et al., 2021)

Our findings independently confirm the major failure mechanisms documented by Krishnapriyan et al.:

1. **Advection Failure Validation:** Our observed collapse at $c = 50$ (Rel $L_2 \approx 1.0$) mirrors Krishnapriyan's core assertion that standard PINNs fail on convection-dominated problems due to soft boundary constraints.

2. **Optimization vs. Capacity Confirmation:** Our supervised regression test ($c = 50$, Rel $L_2 = 0.0208$) provides empirical proof supporting Krishnapriyan's claim that PINN failure is driven by loss landscape non-convexity rather than network capacity limits.

3. **Inefficacy of Loss Weighting:** We confirmed that increasing boundary weights fails to restore convergence, establishing the necessity for advanced optimization strategies (e.g., curriculum learning or sequence-to-sequence causal training).

8.2 Part 2: Project Audit & Reporting Biases (McGreivy & Hakim, 2024)

McGreivy & Hakim (2024) highlighted that *79% of papers claiming neural network superiority compare against artificially weak numerical baselines.* We perform a candid audit of our own benchmark against their criteria:

1. **Honest Limitation — Absence of Numerical Baselines:** Every relative L_2 error in this report was computed against *exact closed-form mathematical solutions.* We did **not** implement traditional Finite Difference or Pseudo-Spectral solvers. Consequently, **we make zero claims of absolute computational supremacy** over classical numerical software.

2. **Embracing Negative Results:** In accordance with McGreivy & Hakim's call for transparent reporting, we highlight our negative results:
    - **FNO Spectral Truncation Failure:** FNO fails completely when evaluated on high-frequency signals ($k \geq 25$, Rel $L_2 > 2.0$), because frequencies beyond $k_{\max} = 16$ are hard-truncated in Fourier space.
    - **PINN Computational Inefficiency:** PINNs are highly inefficient for parametric studies due to their $O(N)$ retraining penalty.

9 Architectural Convergence & Decision Framework

To guide model selection for real-world engineering and scientific applications, we synthesize our empirical findings into a unified decision matrix.

Table 7: Definitive Selection Decision Matrix for Neural PDE Solvers

Constraint Axis	Recommended Model	Primary Advantage	Critical Limitation
Data Availability			
— Equation Only	**PINN**	No ground-truth data needed	High optimization failure rate
— Trajectories Only	**Neural ODE / FNO**	Learns directly from observations	Requires clean training pairs
Problem Scale (N)			
— $1 \leq N \leq 2$	**PINN**	Bypasses dataset creation	Linear $O(N)$ cost scaling
— $N \geq 3$	**FNO**	Millisecond inference	230s dataset generation overhead
Domain / Grid			
— Multi-Resolution	**FNO**	Native resolution invariance	Sensitivity to weight init
— Complex Geometry	**PINN**	Continuous mesh-free co-ords	Boundary gradient instability
— High Frequency ($k \geq 25$)	**PINN / Numerical**	Preserves sharp wavefronts	FNO spectral truncation collapse

A Python Reference Code: Fourier Neural Operator (1D)

The following complete PyTorch implementation was utilized for the 1D Fourier Neural Operator benchmark, including the custom `SpectralConv1d` layer and `FNO1dBlock`.

```python
import torch
import torch.nn as nn
import torch.fft

class SpectralConv1d(nn.Module):
    """1D Fourier Layer: Applies Fast Fourier Transform, filters modes, and returns to physical space."""
    def __init__(self, in_channels, out_channels, modes):
        super(SpectralConv1d, self).__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.modes = modes
        scale = (1.0 / (in_channels * out_channels))
        self.weights = nn.Parameter(scale * torch.rand(in_channels, out_channels, modes, dtype=torch.cfloat))

    def forward(self, x):
        batch_size = x.shape[0]
        # 1. Transform to Fourier domain
        x_ft = torch.fft.rfft(x)
        out_ft = torch.zeros(batch_size, self.out_channels, x_ft.shape[-1], dtype=torch.cfloat, device=x.device)

        # 2. Multiply lower modes with learned complex weights
        out_ft[:, :, :self.modes] = torch.einsum("bix,iox->box", x_ft[:, :, :self.modes], self.weights)

        # 3. Transform back to physical space
        x_physical = torch.fft.irfft(out_ft, n=x.shape[-1])
        return x_physical

class FNO1dBlock(nn.Module):
    def __init__(self, width, modes):
        super(FNO1dBlock, self).__init__()
        self.fourier_conv = SpectralConv1d(width, width, modes)
        self.w = nn.Conv1d(width, width, 1)
        self.activation = nn.GELU()

    def forward(self, x):
        return self.activation(self.w(x) + self.fourier_conv(x))

class FNO1d(nn.Module):
    def __init__(self, modes=16, width=64):
        super(FNO1d, self).__init__()
        self.p = nn.Conv1d(1, width, 1)
        self.layers = nn.Sequential(
            FNO1dBlock(width, modes),
            FNO1dBlock(width, modes),
            FNO1dBlock(width, modes),
            FNO1dBlock(width, modes)
        )
        self.q = nn.Conv1d(width, 1, 1)

    def forward(self, x):
        x = self.p(x)
        x = self.layers(x)
        x = self.q(x)
        return x
```

Listing 1: PyTorch Implementation of 1D Fourier Neural Operator Architecture

B Python Reference Code: Reconciled Measurement Harness

```python
import time
```

```python
import torch
import numpy as np

class ReconciledMeasurementHarness:
    def __init__(self):
        self.registry = {}
        self.device_name = torch.cuda.get_device_name(0) if torch.cuda.is_available()
    else "CPU"

    @staticmethod
    def sync_time():
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        return time.perf_counter()

    @staticmethod
    def relative_l2_error(u_pred, u_exact):
        u_pred_flat = u_pred.flatten()
        u_exact_flat = u_exact.flatten()
        diff_norm = torch.linalg.norm(u_pred_flat - u_exact_flat, ord=2)
        exact_norm = torch.linalg.norm(u_exact_flat, ord=2)
        return (diff_norm / (exact_norm + 1e-8)).item()

    def benchmark_amortized_method(self, name, train_fn, inference_fn, u_exact_test,
    n_test_instances):
        t_start_train = self.sync_time()
        train_fn()
        train_time = self.sync_time() - t_start_train

        t_start_infer = self.sync_time()
        u_pred = inference_fn()
        total_infer_time = self.sync_time() - t_start_infer
        per_instance_time = total_infer_time / n_test_instances

        l2_err = self.relative_l2_error(u_pred, u_exact_test)
        self.registry[name] = {
            "type": "amortized",
            "train_time_sec": train_time,
            "per_instance_sec": per_instance_time,
            "l2_error": l2_err
        }
        return self.registry[name]
```

Listing 2: Benchmarking Harness for Operator vs Instance-Based Method Profiling